МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ "ЛЬВІВСЬКА ПОЛІТЕХНІКА"

ЛЕКЦІЯ 5. ОСНОВИ PYTORCH: ВІД ТЕНЗОРІВ ДО НЕЙРОННИХ МЕРЕЖ

Львів -- 2026

Лекція зі штучного інтелекту 2026-04

Вступ

На попередніх лекціях ми вивчили теоретичні основи навчання з учителем: регресію, класифікацію, функції втрат та градієнтний спуск. Тепер настав час перейти від теорії до практики і познайомитися з **PyTorch** — одним із найпопулярніших фреймворків глибокого навчання.

PyTorch розроблений Meta AI (Facebook) і наразі є частиною Linux Foundation. Він став стандартом як у дослідницькому, так і в індустріальному середовищі завдяки інтуїтивному Pythonic-інтерфейсу та потужній підтримці GPU-обчислень. Якщо ви вже працювали з NumPy — PyTorch здасться вам знайомим, адже він свідомо повторює його API, додаючи автоматичне диференціювання та підтримку GPU.

У цій лекції ми пройдемо весь шлях від базових структур даних до навчання повноцінної нейронної мережі: **тензори** → **операції** → **autograd** → **модель** → **тренування** → **оцінка**.

Теми, що розглядаються

1. Тензори: фундаментальна структура даних PyTorch
2. Перетворення реальних даних у тензори
3. Тензорні операції та функції активації
4. Autograd: автоматичне диференціювання
5. Побудова нейронної мережі з `nn.Module`
6. Повний цикл тренування: від даних до оцінки
7. Ігрові застосування

Тензори: фундаментальна структура даних

Що таке тензор?

Тензор — це узагальнення знайомих математичних об'єктів на довільну кількість вимірів:

Кількість вимірів	Математичний об'єкт	Приклад	PyTorch форма
0	Скаляр	Температура: 36.6	`torch.tensor(36.6)`
1	Вектор	Позиція NPC: [x, y, z]	`shape: (3,)`
2	Матриця	Таблиця статистик гравців	`shape: (100, 5)`
3	3D-тензор	RGB-зображення 224×224	`shape: (3, 224, 224)`
4	4D-тензор	Батч зображень	`shape: (32, 3, 224, 224)`

У PyTorch тензори виконують ту саму роль, що масиви у NumPy, але з двома критичними відмінностями: вони можуть виконуватися на **GPU** та підтримують **автоматичне диференціювання**.

Створення тензорів

PyTorch надає кілька способів створення тензорів:

```
import torch

# Із конкретних значень
damage_values = torch.tensor([150.0, 200.0, 75.0, 300.0])

# Випадкові значення з рівномірного розподілу [0, 1)
random_weights = torch.rand(3, 4)  # Матриця 3×4

# Випадкові значення з нормального розподілу (μ=0, σ=1)
noise = torch.randn(64, 128)  # Типовий шум для нейромережі

# Заповнені нулями або одиницями
zeros = torch.zeros(2, 3)
ones = torch.ones(5)

# Порожній тензор (неініціалізована пам'ять — швидко, але значення "сміттєві")
empty = torch.empty(100, 100)
```

Функція `torch.rand()` генерує значення рівномірно розподілені між 0 і 1 — корисна для ініціалізації ваг. Функція `torch.randn()` генерує значення з нормального розподілу з центром біля 0 — корисна для генерації шуму та тестування. Різницю між їхніми розподілами легко побачити на гістограмі.

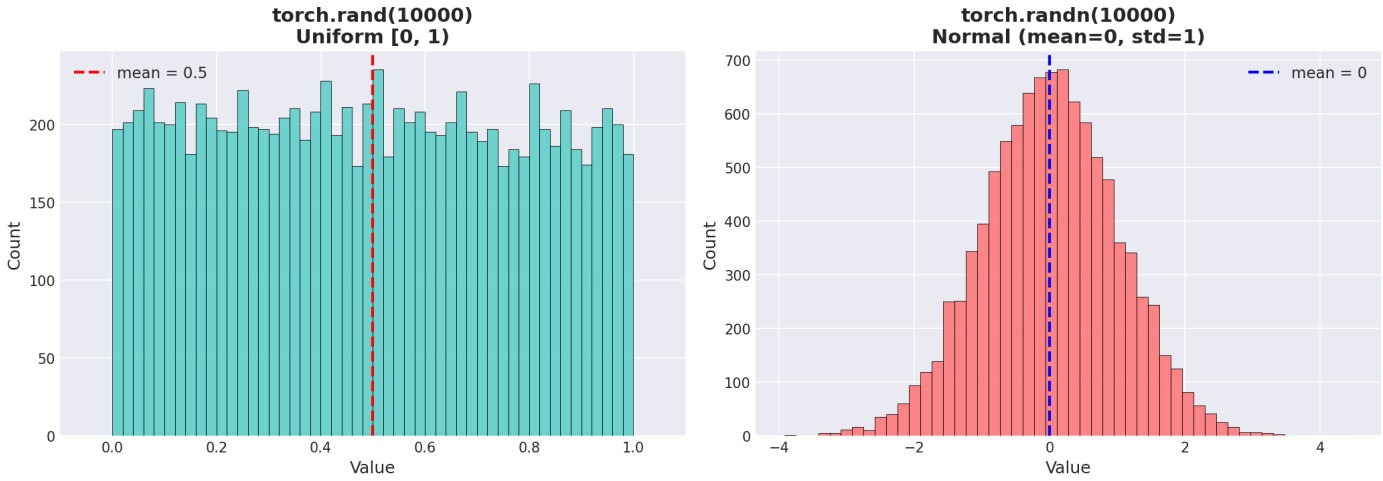

Рисунок 1. Порівняння розподілів значень при різних методах ініціалізації тензорів. `torch.rand()` генерує рівномірний розподіл на відрізку [0, 1), тоді як `torch.randn()` — нормальний розподіл з центром біля 0.

Ключові атрибути тензора

Кожен тензор має три важливі характеристики:

```
t = torch.randn(3, 224, 224)

print(t.shape)   # torch.Size([3, 224, 224]) — форма (розмірності)
print(t.dtype)   # torch.float32 — тип даних
print(t.device)  # cpu (або cuda:0 для GPU)
```

Типи даних визначають точність і використання пам'яті:

Тип	Розмір	Використання
`torch.float32`	4 байти	Стандарт для тренування (за замовчуванням)
`torch.float16`	2 байти	Прискорене тренування (mixed precision)
`torch.int64`	8 байтів	Індекси, мітки класів
`torch.bool`	1 байт	Маски, фільтри

Перетворення реальних даних у тензори

Табличні дані

Реальні дані рідко народжуються як тензори — їх потрібно перетворити. Розглянемо приклад із даними ігрових персонажів:

```
import torch

# Статистики 6 персонажів: [рівень, сила, спритність, інтелект, HP]
characters = torch.tensor([
    [10, 25, 15, 8,  450],
    [15, 18, 22, 20, 380],
    [8,  30, 10, 5,  520],
    [20, 15, 25, 28, 350],
    [12, 22, 18, 12, 420],
    [5,  12, 8,  35, 280],
], dtype=torch.float32)

print(characters.shape)  # torch.Size([6, 5])
# 6 персонажів, 5 ознак кожен
```

Зверніть увагу на `dtype=torch.float32` — ми явно вказуємо тип, оскільки для навчання нейронних мереж потрібні числа з плаваючою комою, а цілі числа (які PyTorch обрав би за замовчуванням для цих даних) не підтримують обчислення градієнтів.

Кодування нечислових даних

Багато реальних даних не є числовими. Для роботи з ними потрібна **попередня обробка** (*preprocessing*):

Текст — кожне слово отримує числовий ідентифікатор:

```
vocabulary = {"attack": 0, "defend": 1, "heal": 2, "dodge": 3}
# Послідовність дій: "attack", "attack", "heal", "defend"
actions = torch.tensor([0, 0, 2, 1])  # shape: (4,)
```

Зображення — представляються як сітка пікселів:

```
# Чорно-біле зображення 28×28 (як MNIST)
grayscale_image = torch.rand(28, 28)  # shape: (28, 28), значення 0–1

# RGB-зображення 224×224 (3 канали: Red, Green, Blue)
color_image = torch.rand(3, 224, 224)  # shape: (3, 224, 224)
```

У PyTorch зображення зберігаються у форматі **CHW** (Channels, Height, Width), а не HWC, як у NumPy/OpenCV. Це зроблено для ефективності згорткових операцій на GPU.

3D-моделі — вершини зберігаються як координати:

```
# Меш з 1000 вершин, кожна має координати (x, y, z)
vertices = torch.rand(1000, 3)  # shape: (1000, 3)
```

Тензорні операції

PyTorch підтримує понад 100 тензорних операцій. Розглянемо найважливіші для побудови нейронних мереж.

Арифметичні операції

```
a = torch.tensor([1.0, 2.0, 3.0])
b = torch.tensor([4.0, 5.0, 6.0])

# Поелементне додавання
c = a + b           # tensor([5., 7., 9.])

# Поелементне множення (НЕ скалярний добуток!)
d = a * b           # tensor([4., 10., 18.])

# Скалярний добуток (dot product)
dot = torch.dot(a, b)  # tensor(32.) = 1*4 + 2*5 + 3*6

# Матричне множення
W = torch.randn(3, 4)  # Матриця ваг 3×4
x = torch.randn(4)     # Вхідний вектор з 4 елементами
y = W @ x               # Вихід: вектор з 3 елементами
# Еквівалентно: y = torch.matmul(W, x)
```

Оператор `@` — це матричне множення, фундаментальна операція нейронних мереж. Коли ми пишемо ˆ = + , у PyTorch це буквально `y = W @ x + b`.

Агрегації

```
scores = torch.tensor([85.0, 92.0, 78.0, 95.0, 88.0])

scores.sum()    # tensor(438.)
scores.mean()   # tensor(87.6)
scores.max()    # tensor(95.)
scores.min()    # tensor(78.)
scores.std()    # tensor(6.5422) — стандартне відхилення
scores.argmax() # tensor(3) — ІНДЕКС максимального значення
```

Функція `argmax()` особливо важлива: саме вона визначає передбачений клас у задачі класифікації — ми обираємо клас з найбільшим скором.

Функції активації

Функції активації вносять **нелінійність** у нейронну мережу. Без них будь-яка послідовність лінійних шарів залишалася б лінійною моделлю (адже добуток матриць — це теж матриця).

```
import torch.nn.functional as F
```

```
z = torch.tensor([-2.0, -1.0, 0.0, 1.0, 2.0])

# ReLU: max(0, z) — найпопулярніша функція активації
relu_out = F.relu(z)      # tensor([0., 0., 0., 1., 2.])

# Sigmoid: 1/(1+e^(-z)) — для ймовірностей
sigmoid_out = torch.sigmoid(z)  # tensor([0.12, 0.27, 0.50, 0.73, 0.88])

# Tanh: (e^z - e^(-z))/(e^z + e^(-z)) — симетрична
tanh_out = torch.tanh(z)  # tensor([-0.96, -0.76, 0.00, 0.76, 0.96])
```

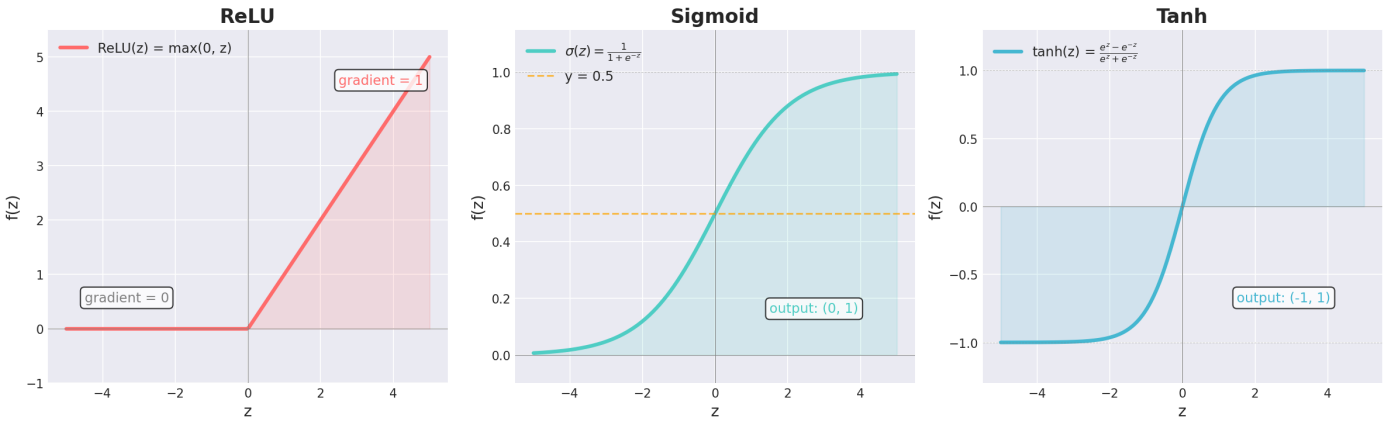

Рисунок 2. Три основні функції активації. **ReLU** — найпростіша і найпопулярніша: обнуляє від'ємні значення, залишає додатні без змін. **Sigmoid** — стискає будь-яке число до діапазону (0, 1), використовується у виходному шарі для бінарної класифікації. **Tanh** — стискає до діапазону (-1, 1), симетрична відносно нуля.

Чому ReLU домінує у прихованих шарах? По-перше, він обчислювально дешевий — лише порівняння з нулем. По-друге, для додатних значень градієнт дорівнює 1, що запобігає проблемі зникаючих градієнтів (*vanishing gradients*), яка мучить sigmoid і tanh у глибоких мережах.

Autograd: автоматичне диференціювання

Навіщо потрібні градієнти?

Пригадайте з лекції 2: градієнтний спуск оновлює параметри моделі за правилом:

$$\leftarrow \quad - \quad \cdot \nabla$$

Для цього потрібно обчислити **градієнт** функції втрат за кожним параметром. У простій лінійній регресії з двома параметрами $(\quad , \quad)$ ми могли обчислити похідні вручну. Але нейронна мережа з мільйонами параметрів робить ручне обчислення неможливим.

Autograd — це система PyTorch, яка автоматично обчислює всі необхідні градієнти. Вам достатньо описати обчислення «вперед» (*forward pass*), а PyTorch автоматично виконає зворотне проходження (*backward pass*) і обчислить усі похідні.

Як це працює: обчислювальний граф

Коли ви виконуєте операції над тензорами з увімкненим відстеженням градієнтів, PyTorch будує **обчислювальний граф** (*computational graph*) — дерево операцій, по якому потім можна «пройтися назад» і обчислити похідні за ланцюговим правилом (*chain rule*).

```
# Простий приклад: f(x) = x²
x = torch.tensor(2.0, requires_grad=True)  # Увімкнути відстеження!
f = x ** 2                                  # f = 4.0

f.backward()     # Обчислити df/dx
print(x.grad)    # tensor(4.) — бо d(x²)/dx = 2x, при x=2 → 4
```

Прапорець `requires_grad=True` каже PyTorch: «стеж за всіма операціями з цим тензором і будь готовий обчислити градієнти». Виклик `.backward()` проходить по обчислювальному графу у зворотному напрямку.

Складніший приклад

Розглянемо функцію кількох змінних:

$$(\quad , \quad , \quad) = \sin(\quad) \cdot \quad ^2 +$$

```
x = torch.tensor(1.0, requires_grad=True)
y = torch.tensor(2.0, requires_grad=True)
z = torch.tensor(0.5, requires_grad=True)

f = torch.sin(x) * y**2 + torch.exp(z)
f.backward()

print(x.grad)  # ∂f/∂x = cos(x)·y² = cos(1)·4 ≈ 2.16
print(y.grad)  # ∂f/∂y = sin(x)·2y = sin(1)·4 ≈ 3.37
print(z.grad)  # ∂f/∂z = e^z = e^0.5 ≈ 1.65
```

PyTorch обчислив всі три часткові похідні автоматично! Для нейронних мереж із мільйонами параметрів — це саме те, що робить `loss.backward()`: обчислює градієнти функції втрат за всіма вагами моделі.

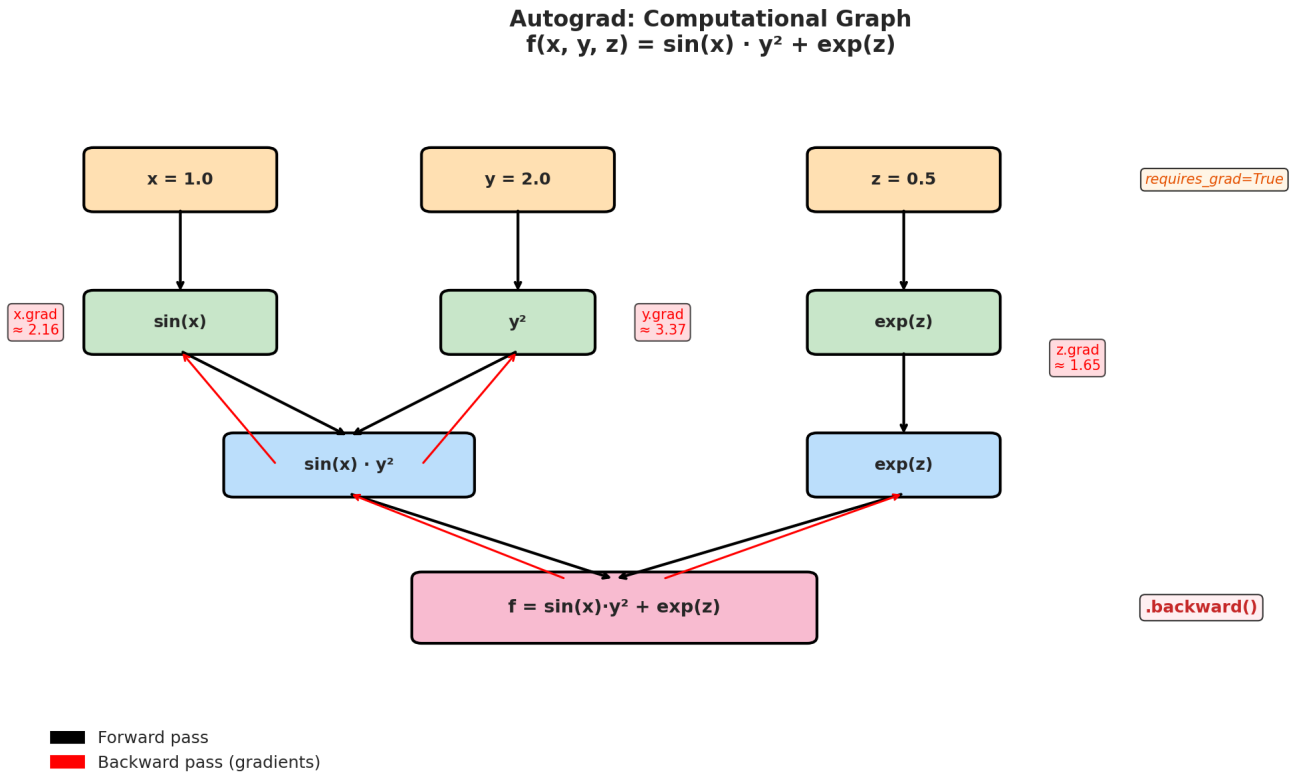

Рисунок 3. Обчислювальний граф для функції $(\ ,\ ,\ ) = \sin(\ ) \cdot\ ^2 +\ $. Стрілки показують напрямок обчислень вперед (forward). Під час `.backward()` PyTorch проходить цей граф у зворотному напрямку, обчислюючи часткові похідні за ланцюговим правилом.

Побудова нейронної мережі з nn.Module

Від теорії до коду

На лекції 3 ми обговорювали нейронні мережі теоретично: шари, ваги, функції активації. Тепер побудуємо мережу у PyTorch, використовуючи модуль `torch.nn`.

Кожна нейронна мережа у PyTorch — це клас, що наслідує `nn.Module`. Вам потрібно визначити дві речі:

1. `__init__` — які шари має мережа (архітектура)
2. `forward` — як дані проходять через ці шари

Приклад: мережа для передбачення ціни будинку

Уявімо задачу регресії: за 87 ознаками будинку (площа, кількість кімнат, район, вік тощо) передбачити його ціну. Це навчання з учителем із MSE як функцією втрат.

```
import torch
import torch.nn as nn

class HousePriceNet(nn.Module):
    def __init__(self):
        super().__init__()
        # Три повнозв'язані шари (fully connected)
        self.layer1 = nn.Linear(87, 64)   # 87 ознак → 64 нейрони
        self.layer2 = nn.Linear(64, 32)   # 64 → 32 нейрони
        self.layer3 = nn.Linear(32, 1)    # 32 → 1 вихід (ціна)
        self.relu = nn.ReLU()

    def forward(self, x):
        x = self.relu(self.layer1(x))  # Шар 1 + ReLU
        x = self.relu(self.layer2(x))  # Шар 2 + ReLU
        x = self.layer3(x)             # Вихідний шар (без активації!)
        return x

model = HousePriceNet()
```

Зверніть увагу: **вихідний шар не має функції активації**. Для регресії нам потрібне необмежене число (ціна може бути будь-якою додатною), а ReLU або sigmoid обмежили б вихід. Для класифікації ми б додали sigmoid (2 класи) або softmax (>2 класів).

Що всередині nn.Linear?

`nn.Linear(in_features, out_features)` — це саме та лінійна модель, яку ми вивчали:

$$= \qquad +$$

де — матриця ваг розміром `(out_features, in_features)`, — вектор зсувів розміром `(out_features,)`.

```
# Подивимося на параметри першого шару
print(model.layer1.weight.shape)  # torch.Size([64, 87])
print(model.layer1.bias.shape)    # torch.Size([64])

# Загальна кількість параметрів моделі
total = sum(p.numel() for p in model.parameters())
print(f"Загалом параметрів: {total}")  # 87*64+64 + 64*32+32 + 32*1+1 = 7777
```

Наша невелика мережа вже має **7 777 параметрів** — і всі вони будуть оптимізовані автоматично за допомогою autograd і градієнтного спуску.

Архітектура мережі

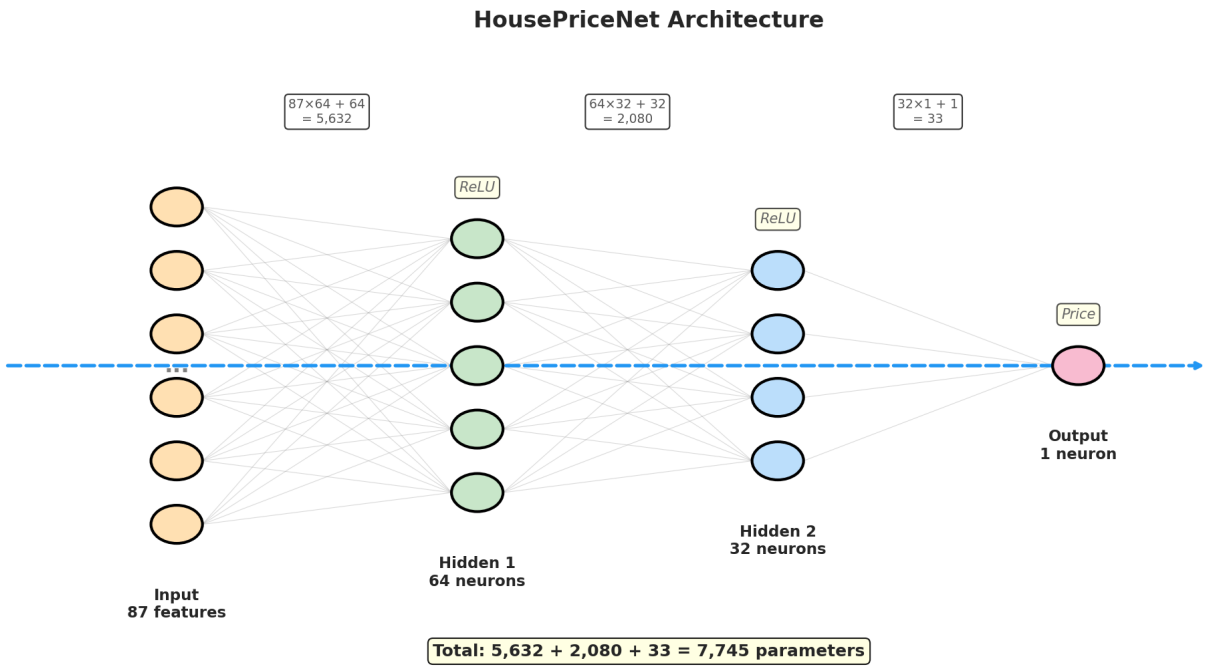

Рисунок 4. Архітектура мережі HousePriceNet. Три повнозв'язані шари з ReLU-активацією у прихованих шарах. Зверніть увагу на «звуження» (87 → 64 → 32 → 1): мережа поступово стискає інформацію від багатьох ознак до одного числа — передбаченої ціни.

Повний цикл тренування

Тепер зберемо все разом: підготовка даних → модель → тренування → оцінка. Це — центральний рецепт PyTorch, який повторюється у кожному проєкті.

Крок 1: Підготовка даних

```
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# Припустимо, data — масив NumPy з 500 будинками, 87 ознак + ціна
# features: (500, 87), targets: (500,)

# Розбиття на тренувальний і тестовий набори (80/20)
X_train, X_test, y_train, y_test = train_test_split(
    features, targets, test_size=0.2, random_state=42
)

# Нормалізація ознак (критично важливо!)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)  # Використовуємо ТІ САМІ параметри!

# Конвертація в тензори PyTorch
X_train = torch.tensor(X_train, dtype=torch.float32)
X_test = torch.tensor(X_test, dtype=torch.float32)
y_train = torch.tensor(y_train, dtype=torch.float32).unsqueeze(1)
y_test = torch.tensor(y_test, dtype=torch.float32).unsqueeze(1)
```

Чому нормалізація критична? `StandardScaler` перетворює кожну ознаку так, щоб її середнє було 0, а стандартне відхилення — 1. Без цього ознаки з великими значеннями (площа: 50–300 м²) домінуватимуть над ознаками з малими (кількість кімнат: 1–5), і градієнтний спуск працюватиме погано — «ландшафт» функції втрат буде витягнутим, а не сферичним.

Важливий момент: `scaler.fit_transform()` для тренувальних даних обчислює середнє та стандартне відхилення і одразу трансформує. Для тестових даних — лише `scaler.transform()` з тими самими параметрами, інакше буде **витік даних** (*data leakage*).

Крок 2: Визначення компонентів тренування

```
model = HousePriceNet()
criterion = nn.MSELoss()            # Функція втрат для регресії
optimizer = torch.optim.Adam(model.parameters(), lr=0.01)
```

Три ключові компоненти:

- **Модель** — наша нейронна мережа з параметрами, які будуть оптимізуватися
- **Функція втрат** (*criterion*) — MSE для регресії, CrossEntropyLoss для класифікації

- **Оптимізатор** — алгоритм оновлення ваг. **Adam** — вдосконалений варіант SGD з адаптивною швидкістю навчання для кожного параметра

Крок 3: Цикл тренування

```
num_epochs = 100

for epoch in range(num_epochs):
    # === Forward pass: обчислюємо передбачення ===
    predictions = model(X_train)

    # === Обчислюємо втрати ===
    loss = criterion(predictions, y_train)

    # === Backward pass: обчислюємо градієнти ===
    optimizer.zero_grad()  # Обнуляємо градієнти з попереднього кроку!
    loss.backward()        # Autograd обчислює ∂L/∂w для всіх ваг

    # === Оновлюємо ваги ===
    optimizer.step()       # w ← w - α·∇L

    if (epoch + 1) % 10 == 0:
        print(f"Epoch {epoch+1}/{num_epochs}, Loss: {loss.item():.4f}")
```

Чому `optimizer.zero_grad()`? За замовчуванням PyTorch **накопичує** градієнти при кожному виклику `.backward()`. Це корисно в деяких ситуаціях (наприклад, при accumulated gradients для імітації великого batch size), але зазвичай нам потрібні свіжі градієнти на кожному кроці. Забудете обнулити — модель не навчиться.

Цей чотирикроковий шаблон — **forward** → **loss** → **backward** → **step** — є серцем тренування будь-якої нейронної мережі у PyTorch, від простої лінійної регресії до GPT.

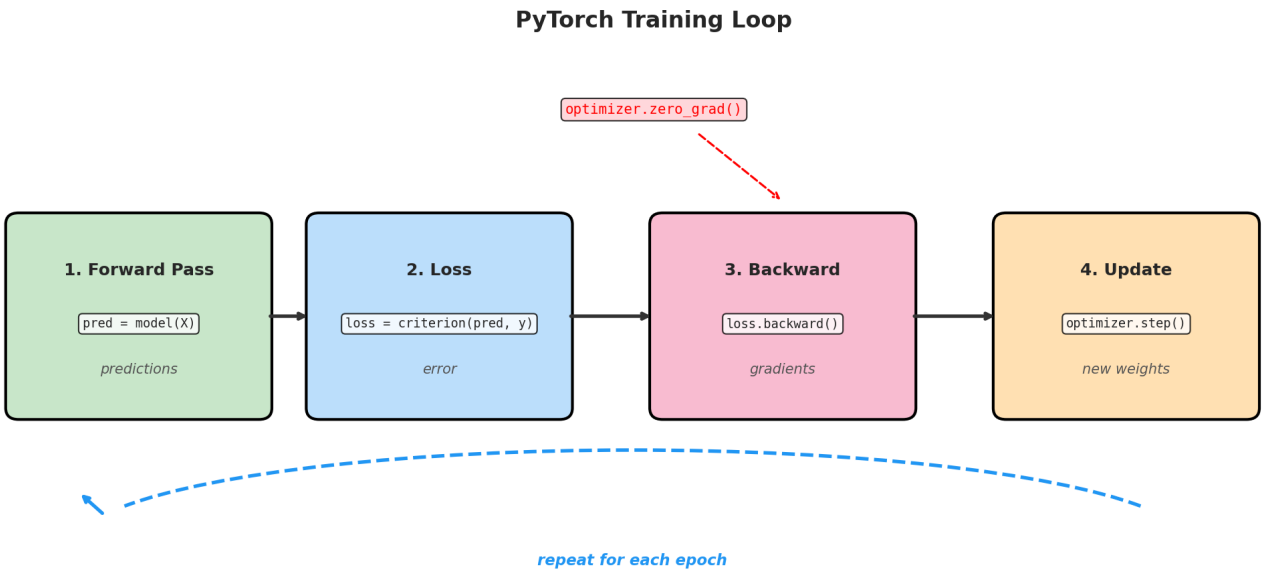

Рисунок 5. Чотири кроки тренувального циклу PyTorch, що повторюються на кожній ітерації. Forward pass обчислює передбачення, функція втрат вимірює помилку, backward pass обчислює градієнти, optimizer оновлює ваги. Цей шаблон є універсальним для будь-якої моделі.

Крок 4: Оцінка моделі

```
# Переключаємо модель у режим оцінки
model.eval()

# Відключаємо обчислення градієнтів (економимо пам'ять і час)
with torch.no_grad():
    test_predictions = model(X_test)
    test_loss = criterion(test_predictions, y_test)

# Обчислюємо метрики
mae = torch.mean(torch.abs(test_predictions - y_test))
print(f"Test MSE: {test_loss.item():.2f}")
print(f"Test MAE: £{mae.item():.0f}")
```

Два важливих моменти:

1. `model.eval()` — переключає модель у режим оцінки. Це впливає на шари, що поводяться по-різному під час тренування та інференсу (dropout, batch normalization).
2. `torch.no_grad()` — відключає autograd. Під час оцінки нам не потрібні градієнти, тому це економить пам'ять і прискорює обчислення.

Крива навчання

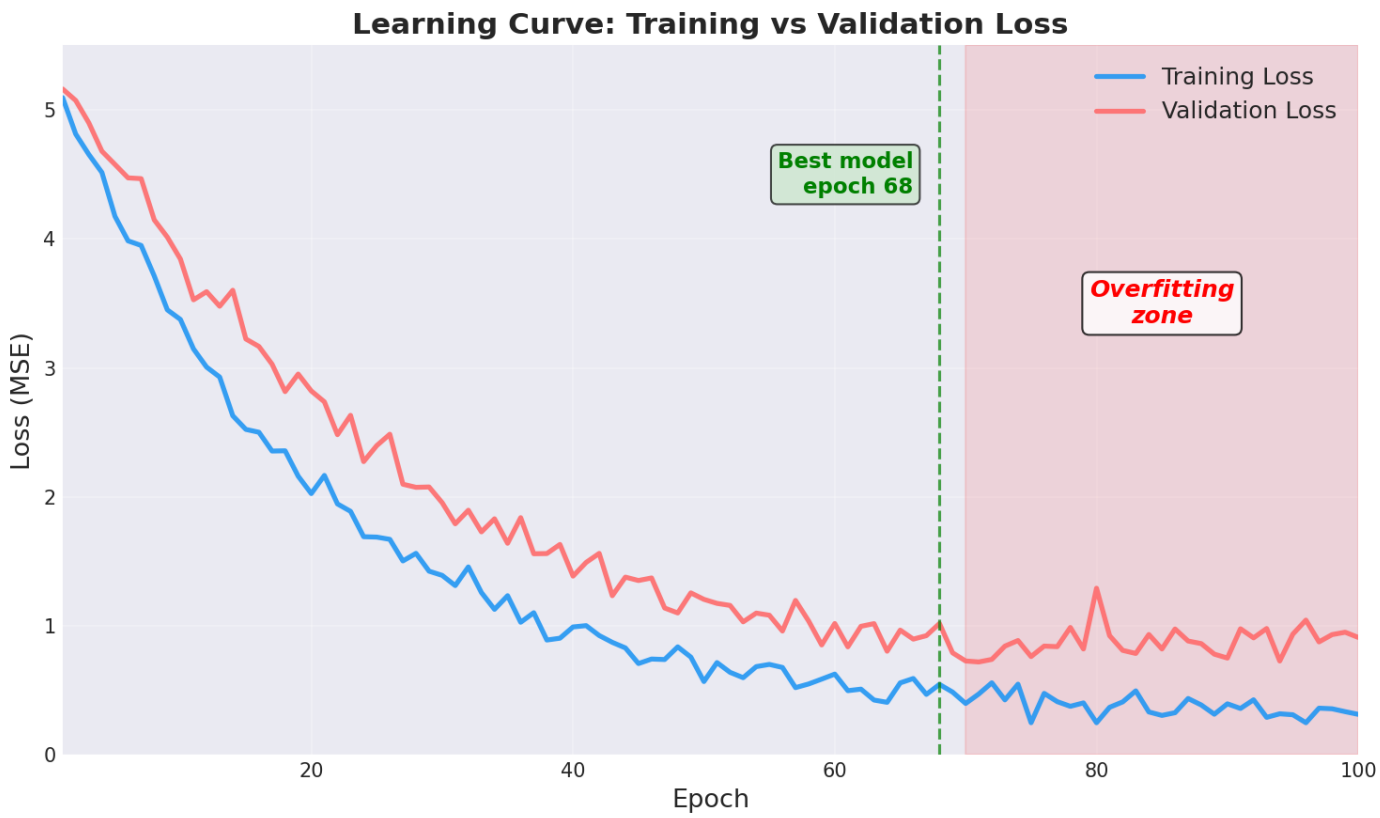

Рисунок 6. Типова крива навчання. Функція втрат швидко падає на перших епохах (модель вловлює основні закономірності), потім поступово виходить на плато (подальше покращення стає все складнішим). Якщо тренувальна крива продовжує падати, а валідаційна зростає — це ознака перенавчання.

Якість даних важливіша за складність моделі

Типова помилка початківця — намагатися покращити результат, ускладнюючи модель (більше шарів, більше нейронів). На практиці **якість ознак** (*feature quality*) та **якість даних** мають значно більший вплив.

Наприклад, для передбачення ціни нерухомості три ознаки — місцезнаходження, площа та стан — дадуть кращий результат, ніж 100 неінформативних ознак з найскладнішою мережею.

Правило великого пальця в ML: **Garbage In — Garbage Out** (сміття на вході — сміття на виході). Перш ніж ускладнювати модель, переконайтеся, що:

1. Дані чисті (немає пропущених значень, помилок, дублікатів)
2. Ознаки інформативні (мають логічний зв'язок з цільовою змінною)
3. Даних достатньо для складності моделі

Ігрові застосування

Балансування шкоди (Damage Balancing)

Задача: передбачити реальну шкоду від удару за параметрами зброї та персонажа.

```
class DamagePredictor(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(8, 32),   # 8 ознак: рівень, сила, тип зброї...
            nn.ReLU(),
            nn.Linear(32, 16),
            nn.ReLU(),
            nn.Linear(16, 1)    # Вихід: передбачена шкода
        )

    def forward(self, x):
        return self.net(x)
```

`nn.Sequential` — зручний спосіб визначити послідовність шарів без написання окремого `forward()`. Корисний для простих мереж.

Класифікація типу гравця

Задача: за поведінковими метриками визначити тип гравця (casual / hardcore / social).

```
class PlayerClassifier(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(12, 32),  # 12 метрик поведінки
            nn.ReLU(),
```

```
            nn.Linear(32, 3)   # 3 типи гравців
        )

    def forward(self, x):
        return self.net(x)

# Функція втрат для багатокласової класифікації
criterion = nn.CrossEntropyLoss()
```

Для класифікації вихідний шар має стільки нейронів, скільки класів, а `CrossEntropyLoss` у PyTorch вже включає softmax — не потрібно додавати його окремо до моделі.

Зв'язок з лабораторними роботами

Все, що ми вивчили в цій лекції, безпосередньо використовується в лабораторних роботах:

Концепція лекції	Де в лабораторній роботі
Тензори	Кожне зображення — тензор `(3, 224, 224)`
Перетворення даних	`transforms.Normalize()` — нормалізація зображень
`nn.Module`	`models.resnet18()` — модель для класифікації
Цикл тренування	Тренування fine-tuned ResNet-18
`model.eval()` + `torch.no_grad()`	Інференс та Grad-CAM візуалізація
`CrossEntropyLoss`	Функція втрат для класифікації зображень
`Adam` / `SGD`	Оптимізатор тренування

Різниця лише у складності моделі: замість простої мережі з 3 шарами у лабораторній роботі використовується ResNet-18 з 18 шарами та ~11.7M параметрів. Але **принцип тренування абсолютно однаковий**: forward → loss → backward → step.

Висновок

У цій лекції ми пройшли повний шлях від фундаменту до практики:

- **Тензори** — універсальний контейнер для даних будь-якої природи: чисел, зображень, тексту, 3D-моделей
- **Тензорні операції** — арифметика, агрегації та функції активації як будівельні блоки нейронних мереж
- **Autograd** — автоматичне диференціювання, яке звільняє від ручного обчислення градієнтів
- **nn.Module** — об'єктно-орієнтований спосіб визначення архітектури мережі
- **Цикл тренування** — forward → loss → backward → step — універсальний шаблон для будь-якої моделі
- **Якість даних** — важливіша за складність моделі

Ці знання є практичним фундаментом для всіх наступних лекцій та лабораторних робіт. Кожна модель, яку ми будемо вивчати далі — CNN, RNN, Transformer — будується на цих самих принципах PyTorch.

На наступній лекції ми перейдемо до **нейронних мереж в деталях**: як з'єднання простих лінійних моделей у шари створює потужні нелінійні моделі, алгоритм backpropagation, та специфічні методи регуляризації для нейромереж — dropout та batch normalization.